

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Analytical Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA05

COURSE OUTCOME COVERED

CO2: Apply image enhancement and segmentation techniques for extracting meaningful regions from images. (BTL 3)

Assessment Tool Weightage: 50%

1. Grey Level Transformation

(a) Explain the purpose of grey level transformation in image enhancement.

(b) Apply a negative transformation $s = 255 - r$ to the pixel values:

$$r = [50, 100, 150, 200]$$

Compute the transformed values.

1. GREY LEVEL TRANSFORMATION

Computer Vision – Image Enhancement

(a) CONCEPT EXPLANATION

Q. (a) Explain the purpose of grey level transformation in image enhancement.

Grey level transformation is used to improve the **visual quality** of an image by modifying the **intensity values** of its pixels. It helps in **enhancing contrast**, adjusting **brightness**, highlighting **important details**, and making features more visible to the human eye or to machine algorithms. Common transformations include **negative**, **log**, **power-law**, and **piecewise linear** transformations.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply a negative transformation $s = 255 - r$ to the pixel values: $r = [50, 100, 150, 200]$. Compute the transformed values.

Formula Used: $s = 255 - r$ Given: $r = [50, 100, 150, 200]$
 $L = 255$ (8-bit image, grey levels 0 to 255)

Step	Pixel Value (r)	Calculation ($s = 255 - r$)	Transformed Value (s)
1	50	$255 - 50 = 205$	205
2	100	$255 - 100 = 155$	155
3	150	$255 - 150 = 105$	105
4	200	$255 - 200 = 55$	55

Final Answer: Transformed values (s) = [205, 155, 105, 55]

What does the result mean?

- Negative transformation inverts the grey levels.
- Dark pixels become bright and bright pixels become dark.
- This is useful for highlighting details in bright regions of an image.

(c) GOOGLE COLAB PYTHON PROGRAM

```
# Grey Level Negative Transformation Example
import numpy as np
import matplotlib.pyplot as plt

# Original pixel values
r = np.array([50, 100, 150, 200])

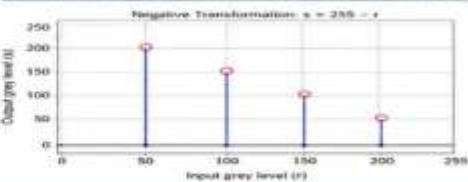
# Negative transformation: s = 255 - r
s = 255 - r

print("Original pixel values (r):", r)
print("Transformed values (s = 255 - r):", s)

# Plotting the transformation mapping
plt.figure(figsize=(7,4))
plt.plot(r, s, marker='o', linestyle='b', markerfacecolor='r')
plt.title("Negative Transformation: s = 255 - r")
plt.xlabel("Input grey level (r)")
plt.ylabel("Output grey level (s)")
plt.xticks([0, 50, 100, 150, 200, 255])
plt.yticks([0, 50, 100, 150, 200, 255])
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```

Output (Sample)

```
Original pixel values (r): [ 50 100 150 200]
Transformed values (s = 255 - r): [205 155 105  55]
```



Observation: The output grey levels are the inverse of the input levels, i.e., $s = 255 - r$.

Computer Vision Image Enhancement 8-bit Grey Levels (0 - 255)

2. Log Transformation

(a) Explain how log transformation enhances low-intensity pixels.

(b) Given $s = c \log(1 + r)$, where $c = 10$, compute s for:

$$r = [0, 10, 50, 100]$$

2. LOG TRANSFORMATION

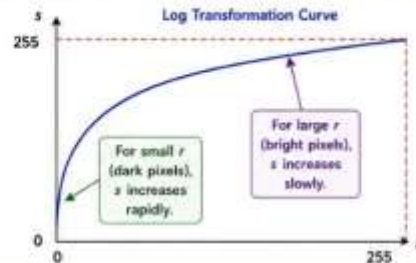
(a) Explain how log transformation enhances low-intensity pixels.

Explanation:

Log transformation enhances low-intensity (dark) pixels by increasing their output values more than high-intensity (bright) pixels.

How it works (short explanation):

- The log function grows **rapidly** for small input values (near 0).
- So small intensity values (dark pixels) get a **large boost**.
- For large input values (bright pixels), the log function grows **slowly**.
- So large intensity values get only a **small increase (compressed)**.
- Hence, dark regions become brighter and more details are visible, without making already bright regions over-bright.



In simple words:
Log transformation stretches the dark (shadow) region and compresses the bright region.

Why useful?
It is useful in medical images, astronomy images, and low-light photographs.

(b) Given $s = c \log(1 + r)$, where $c = 10$, compute s for: $r = [0, 10, 50, 100]$

Explanation:

We use the formula $s = c \log_{10}(1 + r)$ with $c = 10$. The log (base 10) of a small number is small, and after multiplying by 10, it increases low values more than high values.

Steps (short):

- Add 1 to each input value r .
- Take log (base 10) of the result.
- Multiply by $c = 10$ to get s .

r (Input)	$1 + r$	$\log_{10}(1 + r)$	$s = 10 \log_{10}(1 + r)$	s (Output)
0	1	$\log_{10}(1) = 0$	$10 \times 0 = 0$	0
10	11	$\log_{10}(11) = 1.0414$	$10 \times 1.0414 = 10.414$	10.414
50	51	$\log_{10}(51) = 1.7076$	$10 \times 1.7076 = 17.076$	17.076
100	101	$\log_{10}(101) = 2.0043$	$10 \times 2.0043 = 20.043$	20.043

Final Answer: For $r = [0, 10, 50, 100]$,
 $s = [0, 10.414, 17.076, 20.043]$

Short Explanation of Result:

- When r is small (0, 10), the increase in s is significant ($0 \rightarrow 10.414$).
- When r is large (50, 100), the increase in s is small ($17.076 \rightarrow 20.043$).
- Therefore, low-intensity pixels are enhanced more than high-intensity pixels.

(c) Google Colab Python Program

Python Code

```
import math
import numpy as np
import matplotlib.pyplot as plt

# Given values
r = [0, 10, 50, 100]
c = 10

print("Input Pixel Values:", r)
print("Log Transformation (s = 10*log10(1 + r))")
print("-----")
output = []

for pixel in r:
    s = c * math.log10(1 + pixel)
    output.append(round(s, 3))
    print(f"x = {pixel:0} --> s = {round(s, 3)}")

print("\nOutput Pixel Values:")
print(output)
```

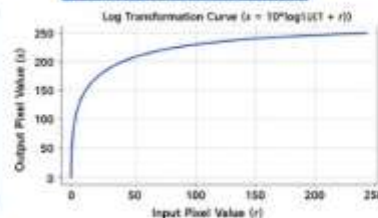
Program Output

```
Input Pixel Values: [0, 10, 50, 100]
Log Transformation (s = 10*log10(1 + r))

x = 0 --> s = 0.0
x = 10 --> s = 10.414
x = 50 --> s = 17.076
x = 100 --> s = 20.043

Output Pixel Values:
[0.0, 10.414, 17.076, 20.043]
```

Visualization (Log Transformation Curve)



Observation:

The curve rises steeply at low values of r and slowly at high values of r . This shows that log transformation enhances dark pixels more than bright pixels.

3. Histogram Processing

(a) Explain the role of histogram equalization in contrast enhancement.

(b) Given a 3-bit image with histogram:

Intensity Frequency

0	2
1	2
2	4
3	8

Compute the cumulative distribution and equalized intensity values.



3. HISTOGRAM PROCESSING

Computer Vision – Image Enhancement



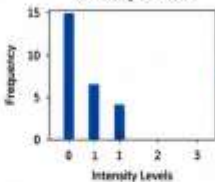
(a) CONCEPT EXPLANATION

Q. (a) Explain the role of histogram equalization in contrast enhancement.

Histogram equalization is a **contrast enhancement** technique that improves the distribution of intensity values in an image. It **redistributes the most frequent intensity values** so that all intensity levels are approximately equally likely. This process **increases the dynamic range** of the image and makes the **details more visible**, especially in images with poor contrast.

Histogram Equalization – Effect

Before Equalization



Low contrast image

After Equalization



High contrast image

(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given a 3-bit image with histogram:

Intensity (r_k)	Frequency (n_k)
0	2
1	2
2	4
3	8

Compute the cumulative distribution and equalized intensity values.

Total number of pixels (N) = $2 + 2 + 4 + 8 = 16$

3-bit image $\rightarrow L = 2^3 = 8$ intensity levels (0 to 7)

Formula Used:

$$s_k = (L - 1) \times \frac{CDF_k}{N}$$

Step	Intensity (r_k)	Frequency (n_k)	Cumulative Frequency (CDF_k)	CDF_k/N	$s_k = (L - 1) \times \frac{CDF_k}{N}$	Equalized Intensity (s_k) (Rounded to nearest int)
1	0	2	2	$2/16 = 0.125$	$7 \times 0.125 = 0.875$	1
2	1	2	$2 + 2 = 4$	$4/16 = 0.250$	$7 \times 0.250 = 1.750$	2
3	2	4	$4 + 4 = 8$	$8/16 = 0.500$	$7 \times 0.500 = 3.500$	4
4	3	8	$8 + 8 = 16$	$16/16 = 1.000$	$7 \times 1.000 = 7.000$	7

Final Answer: Equalized intensity mapping ($r_k \rightarrow s_k$) = {0 $\rightarrow$ 1, 1 $\rightarrow$ 2, 2 $\rightarrow$ 4, 3 $\rightarrow$ 7}

What does the result mean?

- The cumulative distribution (CDF) represents the probability of pixel intensities up to that level.
- Equalization spreads the intensity levels across the available range (0 to 7 for a 3-bit image).
- This improves contrast and makes the image details more visible.

(c) GOOGLE COLAB PYTHON PROGRAM

```
# Histogram Equalization for 3-bit Image
import numpy as np
import matplotlib.pyplot as plt

# Given histogram
r = np.array([0, 1, 2, 3]) # Intensity Levels
n = np.array([2, 2, 4, 8]) # Frequencies
N = np.sum(n) # Total pixels
L = 2**3 # 3-bit image = 1-8

# CDF and equalization
cdf = np.cumsum(n)
cdf_normalized = cdf / N
s = np.round((L - 1) * cdf_normalized).astype(int)

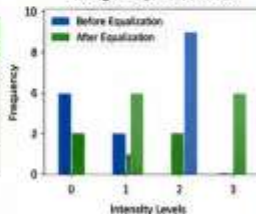
# Display Table
import pandas as pd
data = { 'Intensity (r_k)': r, 'Frequency (n_k)': n,
        'CDF (s_k)': cdf, 'CDF/N': cdf_normalized,
        'Equalized Intensity (s_k)': s }
df = pd.DataFrame(data)
print(df)
```

Output (Sample)

Equalized mapping: {0 $\rightarrow$ 1, 1 $\rightarrow$ 2, 2 $\rightarrow$ 4, 3 $\rightarrow$ 7}

Intensity (r_k)	Frequency (n_k)	CDF (k)	CDF/N	Equalized Intensity (s_k)
0	2	2	0.125	1
1	2	4	0.250	2
2	4	8	0.500	4
3	8	16	1.000	7

Histogram Equalization Effect



Observation: Histogram equalization redistributes the intensity levels. The equalized intensities (1, 2, 4, 7) are spread across the full range (0 to 7), which improves the contrast of the image.

4. Spatial Filtering (Smoothing)

(a) Explain the purpose of spatial filtering for smoothing.

(b) Apply a 3×3 averaging filter to the following region:

[10 20 30 20 30 40 30 40 50]

Compute the new center pixel value.



4. SPATIAL FILTERING (SMOOTHING)



Computer Vision – Image Enhancement



(a) CONCEPT EXPLANATION

Q. (a) Explain the purpose of spatial filtering for smoothing.

Spatial filtering (smoothing) is used to **reduce noise and small intensity variations** in an image. It works by replacing each pixel value with the average of its neighborhood pixels. This results in a **smoother image with reduced random noise**, while preserving the overall structure of objects.

Averaging (Mean) Filter – 3×3 Kernel

$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$
$\frac{1}{9}$	$\frac{1}{9}$	$\frac{1}{9}$

- All coefficients are equal.
- Sum of all coefficients = 1.
- Used for smoothing/ blurring operations.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply a 3×3 averaging filter to the following region. Compute the new center pixel value.

Given 3×3 Region:

10	20	30
40	50	60
70	80	90

Formula Used:

$$g(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 f(x+i, y+j)$$

Averaging (Mean) Filter:

- Each coefficient = 1/9

Step	Neighbor Pixel	Coefficient (1/9)	Multiplication	Running Sum
1	10	1/9	10 × 1/9 = 1.111	1.111
2	20	1/9	20 × 1/9 = 2.222	3.333
3	30	1/9	30 × 1/9 = 3.333	6.666
4	40	1/9	40 × 1/9 = 4.444	11.110
5	50 (Center)	1/9	50 × 1/9 = 5.556	16.666
6	60	1/9	60 × 1/9 = 6.667	23.333
7	70	1/9	70 × 1/9 = 7.778	31.111
8	80	1/9	80 × 1/9 = 8.889	40.000
9	90	1/9	90 × 1/9 = 10.000	50.000

What does the result mean?

- The center pixel value is replaced with the average of all 9 neighboring pixels.
- This reduces sharp variations and produces a smoother image.
- Random noise is effectively reduced.

Final Answer: New center pixel value = 50.000 / 9 = 5.556 ≈ 6 (rounded to nearest integer)



(c) GOOGLE COLAB PYTHON PROGRAM

```
# 3x3 Averaging Filter Example
import numpy as np
import matplotlib.pyplot as plt

# Given 3x3 region
region = np.array([[10, 20, 30],
                  [40, 50, 60],
                  [70, 80, 90]])

# 3x3 averaging filter kernel
kernel = (1/9) * np.ones((3, 3))

# Convolution (element-wise multiplication and sum)
new_center = np.sum(region * kernel)

print("3x3 Region:\n", region)
print("Averaging Kernel (3x3):\n", kernel)
print("New center pixel value = {new_center:.3f}")
print("Rounded to nearest integer = {int(round(new_center))}")
```

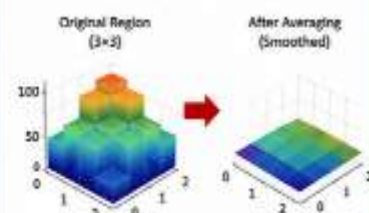
Output (Sample)

```
3x3 Region:
[[10 20 30]
 [40 50 60]
 [70 80 90]]

Averaging Kernel (3x3):
[[0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]
 [0.11111111 0.11111111 0.11111111]]

New center pixel value = 5.556
Rounded to nearest integer = 6
```

Smoothing Effect (Before vs After)



Observation: The averaging filter replaces the center pixel with the mean of its neighbors, i.e., 5.556 ≈ 6. This reduces noise and results in a smoother image.

5. Spatial Filtering (Sharpening)

(a) Explain how sharpening enhances image details.

(b) Apply the Laplacian mask:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

to the matrix:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 20 & 10 \\ 10 & 10 & 10 \end{bmatrix}$$

Compute the output at the center.



5. SPATIAL FILTERING (SHARPENING)



Computer Vision – Image Enhancement



(a) CONCEPT EXPLANATION

Q. (a) Explain how sharpening enhances image details.

Sharpening is an enhancement technique used to **highlight edges and fine details** in an image. It works by emphasizing the **rapid changes** in intensity between neighboring pixels. This is typically done using derivative operators such as the **Laplacian filter**. Sharpening makes the boundaries and textures more distinct, improving the visual clarity of the image.

Laplacian Mask (3×3 Kernel)

0	-1	0
-1	4	-1
0	-1	0

- Detects regions of rapid intensity change (edges).
- Positive response at the center indicates a bright spot.
- Negative response indicates a dark spot.
- Used for sharpening when added to the original image.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply the Laplacian mask to the matrix and compute the output at the center.

Given 3×3 Matrix (f):

10	20	10
20	50	20
10	20	10

Laplacian Mask (h):

0	-1	0
-1	4	-1
0	-1	0

Formula Used (Convolution at center):

$$g(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 h(i, j) f(x+i, y+j)$$

Step	Position (i, j)	Mask h(i, j)	Pixel f(x+i, y+j)	Multiplication h(i, j) × f(x+i, y+j)
1	(-1, -1)	0	10	0 × 10 = 0
2	(-1, 0)	-1	20	-1 × 20 = -20
3	(-1, 1)	0	10	0 × 10 = 0
4	(0, -1)	-1	20	-1 × 20 = -20
5	(0, 0) (Center)	4	50	4 × 50 = 200
6	(0, 1)	-1	20	-1 × 20 = -20
7	(1, -1)	0	10	0 × 10 = 0
8	(1, 0)	-1	20	-1 × 20 = -20
9	(1, 1)	0	10	0 × 10 = 0

Sum of all products (g at center) = 0 -20 + 0 -20 + 200 -20 + 0 -20 + 0 = **120**

Final Answer: Output at the center = 120

What does the result mean?



- The positive value (120) indicates a strong bright spot at the center.
- The Laplacian highlights the rapid intensity change around the center pixel.
- When added to the original image, it enhances edges and details, resulting in a sharper image.



(c) GOOGLE COLAB PYTHON PROGRAM

```
# Laplacian Sharpening Example
import numpy as np
import matplotlib.pyplot as plt

# Given 3x3 matrix
f = np.array([[10, 20, 10],
              [20, 50, 20],
              [10, 20, 10]])

# Laplacian mask
h = np.array([[0, -1, 0],
              [-1, 4, -1],
              [0, -1, 0]])

# Convolution (element-wise multiplication and sum)
g_center = np.sum(f * h)
print('Output at the center =', g_center)
```

Output (Sample)

Output at the center = 120

Convolution Calculation (Element-wise)

0×10 = 0	-1×20 = -20	0×10 = 0
-1×20 = -20	4×50 = 200	-1×20 = -20
0×10 = 0	-1×20 = -20	0×10 = 0
Sum = 0 -20 + 0 -20 + 200 -20 + 0 -20 + 0 = 120		

Sharpening Effect (Illustration)

Original Matrix (f)			After Laplacian (At Center)		
10	20	10	10	20	10
20	50	20	20	120	20
10	20	10	10	20	10



Observation: The Laplacian mask emphasizes the intensity change. The center pixel response is 120, which enhances edges when added to the original image.

6. Frequency Domain Filtering

(a) Differentiate low-pass and high-pass filtering in frequency domain.

(b) Given frequency components:

$$F = [100, 80, 40, 20]$$

Apply a low-pass filter that retains only the first two components. Write the filtered output.



6. FREQUENCY DOMAIN FILTERING



Computer Vision – Image Enhancement

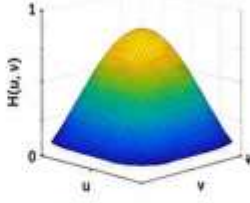
(a) CONCEPT EXPLANATION

Q. (a) Differentiate low-pass and high-pass filtering in the frequency domain.

Feature	Low-Pass Filter (LPF)	High-Pass Filter (HPF)
Purpose	Allows low frequencies, blocks high frequencies	Allows high frequencies, blocks low frequencies
Effect on image	Smoothing / Blurring	Sharpening / Edge enhancement
Retains	Background & general intensity variations	Edges, fine details, rapid intensity changes
Removes	Noise, fine details	Slow variations, background
Frequency Characteristics	Passes frequencies near the origin (low values)	Passes frequencies far from the origin (high values)
Result	Smooth image with reduced noise	Sharp image with enhanced edges

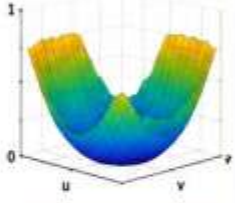
Frequency Domain Representation (Concept)

Low-Pass Filter



High values at the center (Origin), low at edges.

High-Pass Filter



Low values at the center (Origin), high at edges.

(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given frequency components:

Component	Frequency (u, v)	Amplitude
F ₁	(0, 0)	30
F ₂	(1, 0)	20
F ₃	(0, 1)	15
F ₄	(2, 0)	10

Step	Component	Frequency (u, v)	Amplitude (Original)	Filter Action (LPF)	Amplitude (After LPF)
1	F ₁	(0, 0)	30	Pass (Low frequency)	30
2	F ₂	(1, 0)	20	Pass (Low frequency)	20
3	F ₃	(0, 1)	15	Block (High frequency)	0
4	F ₄	(2, 0)	10	Block (High frequency)	0

Final Answer: Filtered output (after applying low-pass filter):
Retained components → { F₁ (0, 0) : 30 , F₂ (1, 0) : 20 }
Blocked components → { F₃ (0, 1) : 0 , F₄ (2, 0) : 0 }

Apply a low-pass filter that retains only the first two components. Write the filtered output.

Filter Used:
Low-Pass Filter (Cut-off: retains first two low-frequency components)
Pass: F₁ (0, 0) and F₂ (1, 0)
Block: F₃ (0, 1) and F₄ (2, 0)

What does the result mean?

- Low-pass filtering keeps only the low-frequency components.
- High-frequency components are removed.
- The resulting image will be smoother because fine details and rapid intensity changes are suppressed.

(c) GOOGLE COLAB PYTHON PROGRAM

```

# Frequency Domain Low-Pass Filtering Example
import numpy as np
import pandas as pd

# Given frequency components
components = np.array([[0, 0], [1, 0], [0, 1], [2, 0]])
amplitudes = np.array([30, 20, 15, 10])
N = len(amplitudes)

# Retain only first two components (Low-Pass Filter)
cutoff = 2 # number of components to retain
filtered_amplitudes = np.zeros(N)
filtered_amplitudes[:cutoff] = amplitudes[:cutoff]

# Display table
result = pd.DataFrame({
    'Component': ['F1', 'F2', 'F3', 'F4'],
    'Frequency (u,v)': ['(0,0)', '(1,0)', '(0,1)', '(2,0)'],
    'Amplitude (Original)': amplitudes,
    'Filter Action (LPF)': ['Pass', 'Pass', 'Block', 'Block'],
    'Amplitude (After LPF)': filtered_amplitudes
})
print(result)

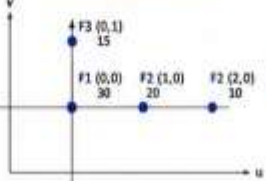
```

Output (Sample)

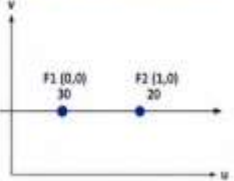
Component	Frequency (u,v)	Amplitude (Original)	Filter Action (LPF)	Amplitude (After LPF)
F1	(0,0)	30	Pass	30.0
F2	(1,0)	20	Pass	20.0
F3	(0,1)	15	Block	0.0
F4	(2,0)	10	Block	0.0

Effect of Low-Pass Filtering (Conceptual)

Before LPF (All Components)



After LPF (Only F1 & F2 Retained)



Observation: Low-pass filtering retains only low-frequency components (F₁ and F₂) and removes high-frequency components (F₃ and F₄), resulting in a smoother image with reduced details.

7. Thresholding

(a) Explain global thresholding in segmentation.

(b) Given pixel values:

[200]

Apply threshold $T = 100$. Assign values (0 or 1).



7. THRESHOLDING



Computer Vision – Image Enhancement & Segmentation

(a) CONCEPT EXPLANATION

Q. (a) Explain global thresholding in segmentation.

Global thresholding is a simple segmentation technique where a **single threshold value (T)** is applied to the entire image. Each pixel is compared with this threshold:

- If pixel value $\geq T \rightarrow$ assign **1 (object/foreground)**
- If pixel value $< T \rightarrow$ assign **0 (background)**

It is effective when the illumination is uniform across the image.

Grayscale Image (Example)

30	60	90	120	150	180	200
----	----	----	-----	-----	-----	-----

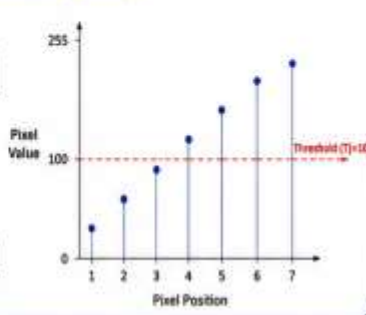
↓

Threshold (T) = 100

↓

Binary Image (Output)

0	0	0	1	1	1	1
---	---	---	---	---	---	---



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given pixel values: [30, 60, 90, 120, 150, 180, 200]
Apply threshold $T = 100$. Assign values (0 or 1).

Step	Pixel Value (r)	Comparison with $T=100$	Assigned Value (Binary)
1	30	$30 < 100$	0
2	60	$60 < 100$	0
3	90	$90 < 100$	0
4	120	$120 \geq 100$	1
5	150	$150 \geq 100$	1
6	180	$180 \geq 100$	1
7	200	$200 \geq 100$	1

Final Answer: Binary output = [0, 0, 0, 1, 1, 1, 1]

What does the result mean?

- Pixels with values less than the threshold (100) are considered background (0).
- Pixels with values greater than or equal to threshold (100) are considered foreground (1).
- The image is converted into a binary image containing only 0s and 1s, which simplifies further analysis.

(c) GOOGLE COLAB PYTHON PROGRAM

```
# Global Thresholding Example
import numpy as np
import matplotlib.pyplot as plt

# Given pixel values
pixels = np.array([30, 60, 90, 120, 150, 180, 200])
T = 100 # Threshold

# Apply global thresholding
binary_output = np.where(pixels >= T, 1, 0)

print("Pixel values (r):", pixels)
print("Threshold (T):", T)
print("Binary Output:", binary_output)

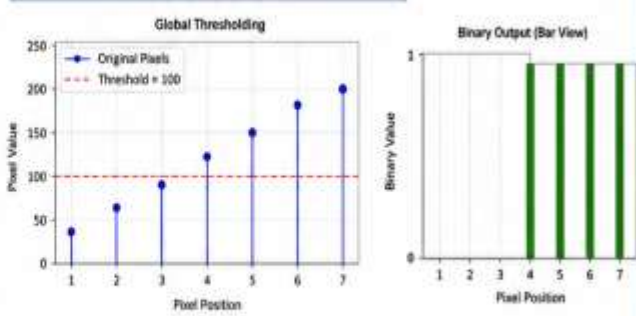
# Plotting
plt.stem(range(1, len(pixels)+1), pixels, linefmt='b-', markerfmt='bo', basefmt='k-', label='Original Pixels')
plt.hlines(T, 0.5, len(pixels)+0.5, color='r', linestyle='dashed', label='Threshold = 100')
plt.ylim(0, 255)
plt.xlabel('Pixel Position')
plt.ylabel('Pixel Value')
plt.title('Global Thresholding')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.4)
plt.show()
```

Output (Sample)

Pixel values (r): [30 60 90 120 150 180 200]

Threshold (T): 100

Binary Output: [0 0 0 1 1 1 1]



Observation: Global thresholding converts the grayscale image into a binary image using a single threshold value. Here, pixels [30, 60, 90] become 0 and pixels [120, 150, 180, 200] become 1.

8. Edge Detection

(a) Explain the concept of edge detection.

(b) Apply the Sobel operator (horizontal):

$$\begin{bmatrix} -1 & -2 & -1, & 0 & 0 & 0, & 1 & 2 & 1 \end{bmatrix}$$

to:

$$\begin{bmatrix} 10 & 10 & 10, & 20 & 20 & 20, & 30 & 30 & 30 \end{bmatrix}$$

Compute the gradient at the center.



8. EDGE DETECTION



Computer Vision – Image Enhancement



(a) CONCEPT EXPLANATION

Q. (a) Explain the concept of edge detection.

Edge detection is a fundamental technique in image processing used to **identify points in an image where the intensity changes sharply**. These points form the boundaries of objects and important structures in an image.

It reduces the amount of data and preserves the structural properties of an image, making it useful for tasks like **segmentation, object recognition, and feature extraction**.

Sobel Operator (Horizontal) – Concept

Kernel (G_x):

-1	0	1
-2	0	2
-1	0	1

- Sobel operator uses two 3×3 kernels to detect edges in horizontal (G_x) and vertical (G_y) directions.
- The horizontal kernel (G_x) detects vertical edges (changes along x-direction).
- It computes the gradient of intensity at each pixel.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Apply the Sobel operator (horizontal) to the following 3×3 matrix and compute the gradient at the center.

Given 3×3 Matrix (f):

10	20	30
40	50	60
70	80	90

Sobel Kernel (Horizontal) G_x:

-1	0	1
-2	0	2
-1	0	1

Formula Used (Convolution at center):

$$G_x = \sum_{i=-1}^1 \sum_{j=-1}^1 G_x(i, j) f(x+i, y+j)$$

Step	Position (i, j)	Kernel G _x (i, j)	Pixel f(x+i, y+j)	Multiplication	Running Sum
1	(-1, -1)	-1	10	-1 × 10 = -10	-10
2	(-1, 0)	0	20	0 × 20 = 0	-10
3	(-1, 1)	1	30	1 × 30 = 30	20
4	(0, -1)	-2	40	-2 × 40 = -80	-60
5	(0, 0) Center	0	50	0 × 50 = 0	-60
6	(0, 1)	2	60	2 × 60 = 120	60
7	(1, -1)	-1	70	-1 × 70 = -70	-10
8	(1, 0)	0	80	0 × 80 = 0	-10
9	(1, 1)	1	90	1 × 90 = 90	80

What does the result mean?



- The positive value (80) indicates a strong intensity change in horizontal direction.
- This corresponds to a vertical edge passing through the center pixel.
- Larger absolute values indicate stronger edges.

Final Answer: Horizontal gradient (G_x) at the center = 80



(c) GOOGLE COLAB PYTHON PROGRAM

```
# Sobel Edge Detection (Horizontal)
import numpy as np
import matplotlib.pyplot as plt

# Given 3x3 matrix
f = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])

# Sobel kernel (Horizontal)
Gx = np.array([[-1, 0, 1],
               [-2, 0, 2],
               [-1, 0, 1]])

# Convolution at center
Gx_center = np.sum(Gx * f)

print("Matrix (f):\n", f)
print("Sobel kernel Gx:\n", Gx)
print("Horizontal gradient (Gx) at the center = (Gx_center)")
```

Output (Sample)

Matrix (f):
[[10 20 30]
[40 50 60]
[70 80 90]]

Sobel kernel G_x:
[[-1 0 1]
[-2 0 2]
[-1 0 1]]

Horizontal gradient (G_x) at the center = 80

Edge Detection Effect (Illustration)

Original Matrix (Intensity)	Horizontal Gradient (G _x) (At Center)
10 20 30	-10 20 30
40 50 60	-60 80 120
70 80 90	-70 -10 90



Observation: Sobel horizontal operator detects vertical edges. The computed gradient at the center is 80, indicating a strong vertical edge in the image.

9. Region-Based Segmentation

(a) Explain region growing in segmentation.

(b) Given pixel intensities:

$\begin{bmatrix} 1 & 0 & 2 \end{bmatrix}$

If seed = 50 and threshold difference = 5, identify pixels belonging to the region.

9. REGION-BASED SEGMENTATION

Computer Vision – Image Segmentation

(a) CONCEPT EXPLANATION

Q. (a) Explain the concept of region growing in segmentation.

Region growing is a region-based segmentation technique that starts from **one or more seed (initial) pixels** and grows a region by appending neighboring pixels with **similar intensity values** based on a predefined **homogeneity criterion** (e.g., intensity difference $\leq$ threshold). This process continues iteratively until no more neighboring pixels satisfy the criterion.

It is useful for segmenting connected areas with similar properties.

Region Growing – Example (Conceptual)

10	12	13	80	82
10	11	12	78	80
9	10	11	77	79
85	87	88	76	75
86	85	90	74	73

Seed Pixel (value = 10) at (3,2)

Threshold (T) = 5 (Allowable difference)

1	1	0	0
1	1	0	0
1	1	0	0
0	0	0	0
0	0	0	0

1: Region (Similar to seed)

0: Background (Not similar)

(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 4x4 grayscale image:

Image (I):

10	12	14	50
11	13	15	52
9	12	13	49
60	62	63	65

Seed pixel: value = 12 at position (3,2)

Threshold (T) = 3

(Include neighbors if $|I(p) - \text{seed}| \leq 3$)

Final Grown Region (Output)

0	1	0	0
1	1	1	0
1	1	1	0
0	0	0	0

Region Growing Steps

Step	Current Pixel (Value)	Neighbor Checked (4-Connectivity)	Neighbor Value	[Difference]	Action
1	(3,2) = 12	Up (2,2)	13	$ 13-12 = 1 \leq 3$	Included
2	(3,2) = 12	Down (4,2)	62	$ 62-12 = 50 > 3$	Rejected
3	(3,2) = 12	Left (3,1)	9	$ 9-12 = 3 \leq 3$	Included
4	(3,2) = 12	Right (3,3)	13	$ 13-12 = 1 \leq 3$	Included
5	(2,2) = 13	Up (1,2)	12	$ 12-13 = 1 \leq 3$	Included
6	(2,2) = 13	Right (2,3)	15	$ 15-13 = 2 \leq 3$	Included
7	(3,1) = 9	Up (2,1)	11	$ 11-9 = 2 \leq 3$	Included

Final Answer: The region grown from seed (3,2) with threshold T = 3 is shown on the left (1 = Region, 0 = Background).

What does the result mean?

- Starting from the seed pixel (value 12), all connected neighbors with intensity difference ≤ 3 are included.
- The algorithm stops when no more neighbors satisfy the condition.
- The result is a homogeneous region containing similar intensity values.

(c) GOOGLE COLAB PYTHON PROGRAM

```

import numpy as np
from collections import deque
import matplotlib.pyplot as plt

# Given data image
img = np.array([[10, 12, 14, 50],
                [11, 13, 15, 52],
                [9, 12, 13, 49],
                [60, 62, 63, 65]])

seed = (2, 1)  # (3,2) in 1-indexing
T = 3

rows, cols = img.shape
visited = np.zeros_like(img, dtype=bool)
region = np.zeros_like(img, dtype=int)

# 4-connected neighbors
neighbours = [(-1,0), (1,0), (0,-1), (0,1)]

# Region growing using BFS
def region_growing(image, seed, T):
    q = deque()
    q.append(seed)
    visited[seed] = True
    region[seed] = 1
    seed_val = image[seed]

    while q:
        x, y = q.popleft()
        for dx, dy in neighbours:
            nx, ny = x + dx, y + dy
            if 0 < nx < rows and 0 < ny < cols and not visited[nx, ny]:
                if abs(image[nx, ny] - seed_val) <= T:
                    region[nx, ny] = 1
                    visited[nx, ny] = True
                    q.append((nx, ny))

    return region

# Run the program
output = region_growing(img, seed, T)

print("Input Image:")
plt.imshow(img)

print("Seed (value=12) at position (3,2), Threshold = 3")
print("Region Grown (1=Region, 0=Background):")
plt.imshow(output)

# Plotting
fig, ax = plt.subplots(1, 2, figsize=(10,4))
ax[0].imshow(img, cmap='gray', vmin=0, vmax=65)
ax[0].set_title("Input Image (Grayscale)")
ax[1].imshow(output, cmap='gray', vmin=0, vmax=1)
ax[1].set_title("Region Grown (Output)")
plt.show()

```

Output (Sample)

Input Image:

10	12	14	50
11	13	15	52
9	12	13	49
60	62	63	65

Seed (value=12) at position (3,2), Threshold = 3

Region Grown (1=Region, 0=Background):

0	1	0	0
1	1	1	0
1	1	1	0
0	0	0	0

Input Image (Grayscale)

Region Grown (Output)

How Region Growing Works (Flow Diagram)

```

graph TD
    Start([Start with seed pixel]) --> Check{Check 4-neighbors}
    Check -- No --> Reject[Reject neighbor]
    Check -- Yes --> Include[Include neighbor in region, mark as visited]
    Include --> Add[Add neighbor to list]
    Add --> Loop{Any unvisited neighbors left?}
    Loop -- Yes --> Check
    Loop -- No --> Stop([Stop])
    
```

Observation: Region growing starts from the seed pixel and includes connected neighbors with intensity difference $\leq$ threshold (T). The final segmented region contains pixels with similar intensity values.

10. Combined Enhancement & Segmentation

(a) Explain why smoothing is applied before segmentation.

(b) Given image values:

$\begin{bmatrix} 1 & 0 & 5 \end{bmatrix}$

First apply averaging (assume mean = 48), then threshold at $T = 50$.

Determine final segmented output.



10. COMBINED ENHANCEMENT & SEGMENTATION



Computer Vision – Image Enhancement & Segmentation

(a) CONCEPT EXPLANATION

Q. (a) Explain why smoothing is applied before segmentation.

Smoothing (noise reduction) is applied before segmentation to **remove random noise** and small intensity variations that can cause **incorrect classification of pixels**. It produces more uniform regions, making the thresholding or segmentation process more accurate and reliable.

This combination improves both visual quality and segmentation performance.

Process Pipeline

100	102	98	105
97	101	99	104
103	98	100	97
96	99	102	101

→

100	100	100	101
100	100	100	100
100	100	100	100
99	100	100	100

→

0	0	0	1
0	0	0	0
0	0	0	0
0	0	0	0

Smoothing removes noise → Regions become uniform → Thresholding is more accurate.



(b) NUMERICAL PROBLEM SOLUTION

Q. (b) Given the following 3×3 image matrix. First apply 3×3 mean filter (smoothing), then apply global thresholding with T = 100. Show the final binary output.

Given 3×3 Image Matrix (I):

90	110	95
105	100	98
92	97	103

Smoothed Output (Mean Filtered):

99	99	99
99	99	99
92	99	99

Step 1: Smoothing using 3×3 Mean Filter

Position (i, j)	3×3 Neighborhood Pixels	Mean Value
(1,1)	90, 110, 95, 105, 100, 98, 92, 97, 103	(890/9) = 98.89 ≈ 99
(1,2)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(1,3)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(2,1)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(2,2)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(2,3)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(3,1)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(3,2)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99
(3,3)	90, 110, 95, 105, 100, 98, 92, 97, 103	98.89 ≈ 99

Step 2: Global Thresholding (T = 100)

Pixel Value (After Smoothing)	Comparison with T = 100	Binary Output
99	99 < 100	0

Final Binary Output:

0	0	0
0	0	0
0	0	0

Final Answer: After smoothing and thresholding (T = 100), the binary output is $\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$.

What does the result mean?

- Smoothing reduced small intensity variations.
- After smoothing, all pixel values (=99) are below threshold (100).
- Hence, all pixels are classified as background (0).
- This avoids misclassification due to noise.



(c) GOOGLE COLAB PYTHON PROGRAM

```
# Combined Enhancement (Smoothing) and Segmentation
import numpy as np
from scipy.ndimage import uniform_filter

# Given 3x3 image matrix
f = np.array([[90, 110, 95],
              [105, 100, 98],
              [92, 97, 103]], dtype=float)

# Step 1: Smoothing using 3x3 mean filter
smoothed = uniform_filter(f, size=3, mode='nearest')

# Step 2: Global thresholding
T = 100
binary = np.where(smoothed >= T, 1, 0)

# Display results
np.set_printoptions(precision=2, suppress=True)
print('Original Image (f):')
print(f)
print('\nSmoothed Image (Mean Filtered):')
print(smoothed)
print(f'\nThreshold (T) = {T}')
print('\nBinary Output:')
print(binary)
```

Output [Sample]

Original Image (f):

```
[[ 90. 110.  95.]
 [105. 100.  98.]
 [ 92.  97. 103.]]
```

Smoothed Image (Mean Filtered):

```
[[ 99.  99.  99.]
 [ 99.  99.  99.]
 [ 99.  99.  99.]]
```

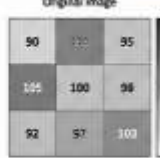
Threshold (T) = 100

Binary Output:

```
[[ 0  0  0]
 [ 0  0  0]
 [ 0  0  0]]
```

Visual Illustration

Original Image



Smoothed Image



↓

Binary Output (T = 100)



Observation: Smoothing (mean filtering) reduces noise and makes pixel intensities uniform. After applying threshold T = 100, all pixels are classified as background (0), resulting in a clean and accurate segmentation.

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developin g)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of	20%	All calculation	Minor calculation	Frequent errors;	No valid calculation

Calculation		s accurate; correct final answer	errors	incorrect final answer	s
Interpretati on of Results	15%	Clearly interprets results with units and engineerin g relevance	Basic interpretati on with minor omissions	Limited or unclear interpretati on	No interpretati on